Atmospheric Simulation Chamber: Revision and Future Scheme

Summer project 2025,
Samu Honkakoski,
honkasam@uef.fi
University of Eastern Finland
Department of Technical Physics
Aerosol Physics Research Group

Atmospheric simulation chamber upgrade was started by Samu Honkakoski with supervision of Arttu Ylisirniö and Pasi Miettinen. The aim of this project is to create an easy-to-use and safe simulation chamber for its users. The project is huge, so one summer wasn't enough time to finish it totally. It is safe to say that the project is about halfway through.

This document is initially written for the use of Aerosol physics group at University of Eastern Finland. It contains detailed information about atmospheric simulation chamber equipment, working principles and operations under the hood.

AI usage

AI aid has been used in all the code that includes implementing different communication protocols. The framework of these Python scripts has been done with the aid of Microsoft Copilot, implementing the script to the project needs has been done by me. AI has not been used for writing this report.

Contents

1 Introduction **1**
1.1 Automatic Data Logger 1
1.2 Automatic Process Control (Incomplete) 1

2 Software **2**
2.1 Spyder (Python) 2
2.2 PgAdmin 2
2.3 Grafana 2
2.4 Arduino IDE 3

3 Devices and connection **3**
3.1 Mainframe Computer 3
3.2 Devices 3
3.3 Connections 5
3.4 Hardware connection standards 6
3.5 Communication protocols 7
3.5.1 ASCII protocol 7
3.5.2 Modbus RTU 8
3.5.3 Modbus TCP/IP 10

4 Discussion and Future Scheme **11**
4.1 Automatic data logger 11
4.2 Automatic process control 11

References **13**

1 Introduction

Atmospheric simulation chamber of Aerosol physics research group in University of Eastern Finland is in need of an update. Before this project work, every device was manually adjusted and most of the data was logged to individual data files. The update gives a chance for more efficient, more prompt, and most importantly, better resolution and accuracy for the measuring campaigns. The chamber update project has two modules: Data logger module and process control module.

1.1 Automatic Data Logger

The idea behind data logger module is that all the essential data of simulation chamber is saved into one database. The data is imported from database to visualization software for visual examination. In the visualization software, one can observe larger time series data and also near real time data. Data logging requires a connection between measurement instrument and the mainframe computer that controls the instruments and data acquired from the instruments. Data transmission requires also a protocol how the data is transmitted.

There are multiple ways transmit data from device to other, but this project requires robust transmission. The data logger uses couple of connection standards and also above that the communication protocols. The choices were serial communication with D-Sub connectors and networking with Ethernet connection.

After the introduction to process control automatization, in section 2 all the software that is used in this project (so far) is presented. Section 3 goes deeper into how the different measurement instruments and devices are connected to the mainframe computer. Section 4 is for discussion and the future intentions and scheme of this project.

1.2 Automatic Process Control (Incomplete)

The objective of the automatic process control is to create steady-state simulation conditions to run atmospheric simulation studies. Process control includes the regulation of pressure and relative humidity (RH) inside the simulation chamber. Condition regulation demands collaboration of RH sensors, pressure sensors, mass flow controllers and exhaust fan.

Another important part of automatic process control is working safety. UV lights can be used for studying how certain aerosol particles and gas vapours react in the atmosphere. However, UV lights can be hazardous to human eyes and skin. The high exposure of UV light can cause skin cancer or photokeratitis for the eyes. This gives us high motivation to automate and remotely control UV lights inside the chamber room.

It is crucial to know that automatic process control project is unfinished. More about the plans are discussed in section 4.

2 Software

Software used in this project was Python implemented on Spyder, pgAdmin, Grafana and Arduino IDE. These platforms were used because these are easy-to-use for further work and they have the properties needed in the upgration.

2.1 Spyder (Python)

Spyder is an open source Python development environment designed for science and data analysis. It fuses together the advantages of popular Python IDEs such as Visual Studio Code (VSCode) and Jupyter. One can easily see the created parameters and use all the most common modules available for Python. There are several modules designed for Python which are used in this project, e.g. modules for connections and communication protocols. We will discuss about these protocols in section 3.

Python program is also used for saving data into PostgreSQL database. There are several modules for saving data into PostgreSQL database, but we used module *psycopg2*. It is crucial to know what kind of data is being saved into the database before using Python script for it. In the Python program is also a function that saves data into text file as a backup.

2.2 PgAdmin

PgAdmin is administration and development platform for PostgreSQL, a database management system. The database management system is mainly used for saving data from measurement devices and visualizing time series data in Grafana. PgAdmin is easy-to-use administration platform, where one can control the parameters and tables in which the data is saved. One can add or remove columns what is needed or not.

As mentioned earlier, it is crucial to know what data types we can save into the database tables before adding or removing tables. If one is doing significant changes into the database, it should be changed in the Python program to avoid any errors in script.

2.3 Grafana

Grafana is observability framework for data visualization. Grafana is operated through dashboards, where various visual graphs can be added. In this project, Grafana is used for visualiz-

ing real-time and larger interval time series data. There, one can see data from Thermo Fisher analyzers, the ADAM analog module, and Vaisala RH sensors.

In the upgrade of the atmospheric simulation chamber, Grafana utilizes data obtained from PostgreSQL. Grafana has a property that one can easily add PostgreSQL as the source of visualized data. Our goal is to visualize data effortlessly to check if the Python program produces data correctly. Data can be seen real-time, so it is easier to fix problems just when they occur.

2.4 Arduino IDE

Arduino is an open-source hardware and software developer that manufactures microcontrollers. Arduino IDE is a programming environment for Arduino microcontrollers. One can program and upload a script for several Adafruit and Arduino devices to run on these devices when they are connected to a power source. In this project, I prepared an Arduino program for controlling older versions of Alicat mass flow controllers using serial commands. The test device was Arduino UNO R3.

3 Devices and connection

3.1 Mainframe Computer

All of the programming and other operations for the chamber upgrade were done with *Giada F302*, an industrial computer. The computer has *Ubuntu 24.04 LTS* as an operating system and it runs several applications for the chamber to work properly. The computer runs a DHCP server (*Dynamic Host Configuration Protocol*), which dynamically creates IP addresses (*Internet Protocol*) for every device connected to the computer with an RJ45 Ethernet cable.

The computer also runs a relational database with PostgreSQL, a database management system, and along with the database is a Grafana server. PostgreSQL is used with pgAdmin, which is an administration and development platform for PostgreSQL.

In this project, the main focus is to make a visual representation of time series data for atmospheric gas concentrations, temperature, and relative humidity (RH). These parameters are of interest in the automatic data logger. Now, a brief demonstration of the devices and measurement instruments used in this project besides the computer.

3.2 Devices

Thermo Fisher analyzers

Thermo Fisher 42i-TL is trace level chemilumenescence NO-NO_2-NO_x (later referred as NO_x) analyzer. It uses infrared light emissions, ozone, molybdenum converter, solenoid valve

and flow sensor to analyze NO_x concentration [1].

Thermo Fisher 43i-TLE is enhanced trace level pulsed fluorescence SO_2 analyzer. It detects SO_2 concentration by pulsing ultraviolet light emissions that SO_2 absorb and become excited in one wavelength. After that, it decays to a lower energy state that emits UV light at a different wave length. This is an efficient method to measure SO_2 concentration [2].

Thermo Fisher 49i is UV photometric O_3 analyzer. It operates in a principle that ozone molecules absorb UV light at a wavelength of 254 nm [3]. The degree to which UV light is absorbed is an direct problem of Beer-Lambert law

$$\frac{I}{I_0} = e^{-\mathrm{KLC}} \tag{3.1}$$

where I is UV light intensity of sample gas (O_3), I_0 is UV light intensity of reference gas, K is molecular absorbtion coefficient (308 cm^{-1} at STP conditions), L is length of cell and C is the ozone concentration in ppm [3].

The project task is to transfer NO_x, SO_2, and O_3 data into the mainframe computer through a wired connection. Thermo Fisher analyzers have multiple built-in data transferring protocols for remote operations. The most significant protocols for this project are the ASCII protocol and Modbus TCP/IP (which are discussed later in section 3). In user manuals ASCII protocol is referred as C-Link protocol [1, 2, 3]. All the Thermo Fisher devices have the same communication protocols used in data transfer.

Advantech ADAM devices

ADAM-4520 isolated RS-232 to RS-422/485 converter is an automatic RS-485 data flow controller used as a connecting piece for **ADAM-4117** analog-to-digital converter and the mainframe computer. ADAM-4117 is a robust device for converting analog voltages into digital signals to be later converted into meaningful data. ADAM-4117 has 8 differential channels that can take up to 4 analog devices to send signal forward. For this project ADAM devices are used for transmitting Vaisala analog RH sensor data. ADAM-4117 transmit voltage signal in 16-bit unsigned integer. This means that the maximum value for transmitted voltage is 65535. More information about ADAM-4117 and ADAM-4520 can be found in datasheets [4, 5].

Vaisala HMP60/110 analog RH sensor

Vaisala HMP60/110 series analog sensor is linearly calibrated relative humidity (RH) sensor. The sensors can measure temperature from -40 °C to $+80$ °C and it measures and calculates RH and dew/frost point. The output voltages in the devices go from 0 V to 1 V, 2.5 V or 5 V.

With further investigation, we can see that output voltage of Vaisala device isn't relevant for temperature and RH conversion when using ADAM-4117 analog module.

Vaisala sensor's analog signal can be converted into correct units by a simple linear transformation. This information combined with ADAM-4117 signal transmission type gives us an easy way to convert voltage signal into temperature and RH with simple equation of straight line from minimum voltage to maximum voltage

$$\begin{aligned} RH &= \frac{100\ \%}{65535}x \\ T &= \frac{80\ ^\circ\mathrm{C} - (-40\ ^\circ\mathrm{C})}{65535}x - 40\ ^\circ\mathrm{C}, \end{aligned} \tag{3.2}$$

where x is output voltage from Vaisala device in unsigned integer value. More information about Vaisala sensors can be found from Vaisala product documentation web page [6].

3.3 Connections

There are multiple ways to create a connection between two or more electronic devices. The great separating line would be wireless (e.g. Bluetooth, WLAN, NFC) and wired (e.g. USB, D-Subminiature, Ethernet) connections. In this project, the main focus is on wired connections and more precisely, serial communication with USB, Ethernet (RJ45) and D-Sub connectors (DB-9 and DB-15).

D-Sub and RJ45 connectors are the main connector types in this project work. D-sub connectors such as DB-9 and DA-15 are widely used in older devices and can be ordered from manufacturers if needed.

There are a few important parameters and features that are not included in hardware connection standards or communication protocols sections. **Baud rate** appears in many user and reference guides. Baud rate very important concept and it defines the speed of signal modulation that happens between connected devices. For the hardware connections this report explains, the baud rate is bits transmitted per second. Baud rate must be paired for the connected devices for transmission bus to work properly. If baud rate is mismatched, the devices can't communicate with each other.

Other important parameter to know is port. Port can be defined as the sorting system of device network traffic. It defines the application that the device connection is used for. This is used in networking standards and lies above the Internet protocol and communication network.[7] Like baud rate, port must be paired on all involved devices.

3.4 Hardware connection standards

If there are two or more connected devices, there is also a hardware connection standard used for information transportation between devices. Here are the most commonly used hardware connection standards in this project.

RS-232

RS-232 is one of the two hardware connection standards used in this project for serial communication. It is worth diving into how the RS-232 standard operates and how the information transfers between devices. RS-232 standard is for bit-wise information exchange between data terminal equipment (DTE) and data communication equipment (DCE). In other words, RS-232 standard can exchange information one bit at a time. RS-232 communication operates with simplex. This means that simultaneous information exchange isn't possible.[8]

Information exchange in RS-232 happens with positive and negative voltages. Voltages from -15 V to -3 V represent logical value 1 and voltages between 3 V and 15 V represent logical value 0. As mentioned before, devices can send information with this standard only one bit at a time and with simplex information exchange [8]. This is possible with asynchronous communication. Asynchronous serial communication is a communication method that uses start and stop bits to identify the message that is shared between the the signal sender and receiver. It doesn't need shared clock system to work. Common standard communication protocols that utilize asynchronous communication include RS-232, RS-422 (not discussed in this report), and Universal Asynchronous Receiver/Transmitter (UART) protocol [9].

RS-485

RS-485 standard is based on differential signaling of a pair of impedance-matched signal wires. Let's call these wires A and B. Differential signaling is based on signal transmitter and receiver creating a voltage difference between the wires A and B. Logical state 1 is reached when voltage on wire A exceeds the voltage of wire B [10]. It is possible to send signals simultaneously from connected parties but it creates complexity in signaling that is unnecessary for this project. The need for this project is just for the computer to send a signal to AD-converter to get back data signals.

Ethernet connection

RJ-45 Ethernet connector with Cat5e standard can provide up to 100 MHz performance. Cat5 signaling is based on differential signaling. While proper understanding of different networking standards is interesting and rather complex topic, it is out of scope of this project work. It is enough to know that computer networking can be used as a data transfering method.

3.5 Communication protocols

When devices are connected with wireless or wired connections, the communication between devices needs a communication protocol for information to be transferred between devices. There are several communication protocols, which come in handy in different scenarios and device connections.

3.5.1 ASCII protocol

ASCII protocol is a query-response communication protocol which uses ASCII characters when transmitting and receiving messages. It's standardized protocol with command configuration but every device manufacturer has their own commands for data transmission. An example of ASCII protocol command sequence can be examined in table 1.[11] However, usually ASCII commands don't require implementing the full command sequence for calling the devices. The usual ASCII command sequence would be the address of the device, command and command termination (carriage return). [12].

Table 1: ASCII command sequence. The command sequence has the same structure regardless the direction of the information flow, except the response from slave devices does not have *command* line. Source: Brainboxes [11]

#	**AA**	**00**	**(Data)**	**[CS]**	**(CR)**
Delimiter	Address	Command	Data	Checksum	Carriage Return

Let's go through the meaning of each line of the command sequence introduced in table 1.

1. # – Delimiter

   Delimiter is used to signal the receiver what kind of command is it receiving.

   Delimiter can be one of these characters:

   # , % , $, @ , $\sim$

2. AA – Address

   Address signals that which receiver device the message is meant for. Address is usually 2 ASCII characters

3. 00 – Command

Contains the command that you want to send to the receiver. Command is also usually 2 ASCII characters.

4. Data

   When the host device sends data frame, it requests data from slaves. When slaves send data frame, it contains the actual data requested.

6. [CS] – Checksum

   An optional two character command for detecting communication errors. Checksum helps the host and the slaves detect corrupted command strings. When checksum is enabled, without valid checksum, the command will be ignored. This project doesn't use checksum.

6. (CR) – Carriage Return

   Carriage return terminates the command.

This command sequence is implemented in most of the Python modules for my knowledge. In other words, there is no need to use this whole sequence in Python scripts. It is enough to know the command line and data line to send messages to receiving devices.

ASCII protocol was first introduced for RS-485 connection standard, but it can be used widely in other connection standards [11]. In this project, ASCII protocol is utilized through Ethernet RJ45 networking standard. It is important to note that RJ45 networking standard utilizes differential signaling as well as RS-485, so ASCII protocol can be easily implemented into network connections.

ASCII Protocol in data logger

Thermo Fisher analyzers were connected to mainframe computer by RJ45 networking standard. This means that Thermo Fisher analyzers were in a local network and has IP addresses as device identifier. The task was to identify which device is under which IP address. ASCII (C-Link) protocol was used for obtaining device information from Thermo Fisher devices. It was the only way to obtain basic information data from the devices. The gained information was implemented in text files as the first line of each text file. This way it is easier to recognize where the data came from.

3.5.2 Modbus RTU

Modbus RTU protocol is a query-response protocol, just like ASCII protocol. Modbus RTU has very similar information transmission technique, but the main difference is how the transferred data is packed. As in ASCII protocol the data is transferred one bit at a time as ASCII

characters, Modbus uses 8-bit registers to pack the data for transmission. The host device sends a query to a specific slave device to send data packed into registers. The slave device responses with communication sequence. Complete communication sequence is presented in table 2. [13]

Table 2: Modbus RTU command sequence. Source: Modbus Protocol Reference guide [13].

START	ADDRESS	FUNCTION	DATA	CRC CHECK	END
3.5 char time	1 byte	1 byte	$n \times 1$ bytes	2 bytes	3.5 char time
-	0-247	Read/Write	-	Error Check	-

As one can see, the structure of command sequence in Modbus RTU is very similar to ASCII protocol command sequence. Let's go through each line of the command sequence.

1. START/END

   Messages start and ends with silent interval of minimum 3.5 character times.

2. ADDRESS

   The host PC derives unique addresses for each slave devices. Slave addresses can be of range 1-247. Address 0 sends the message to all connected slaves.

3. FUNCTION

   Function line defines what action the host requests from the slave. It can reading from registers or writing new information to registers. Function is 8-bit (1 byte) command and can be integer from 0 to 255.

4. DATA

   Data line tells the slave what data is requested. Data can be of size $n \times 1$ bytes. since registers are of size 1 byte, slaves can have multiple registers that holds the data.

5. CRC CHECK

   Error checking field based on Cyclical Redundancy Check (CRC). CRC check contains 16 bits (2 bytes) of binary data. It is calculated two times, first time when the host requests data, and second time when data is received. If CRC value of received data is not equal to request CRC value, error occurs.

Modbus RTU function formats are defined universally. It contains register addresses and what can be done for the register. The registers can be coil, input status, input registers and holding registers. The most significant function format for data logging holding registers. The holding registers hold the actual measured data from the slave device. Holding registers are 16-bit registers, and the register values can be read or written.[13]

Modbus RTU Protocol in data logger

Modbus RTU is used in serial communication, at least in data logging of the project. ADAM-4117 analog module has a built-in Modbus channel, which was used to collect Vaisala RH sensor data. More information regarding the using of ADAM-4117 with Modbus can be found from ADAM-4100 series user manual [14].

3.5.3 Modbus TCP/IP

Modbus TCP/IP is very similar to Modbus RTU but it operates through networking standards. This implicates that the command sequence differs from Modbus RTU command sequence. Modbus TCP/IP command sequence is presented in table 3.

Table 3: Modbus TCP/IP command sequence. Source: Modbus Protocol Reference guide [13].

TRANSACTION IDENTIFIER	**PROCOTOL IDENTIFIER**	**LENGTH**
Bytes 0-1	Bytes 2-3	Bytes 4-5
UNIT IDENTIFIER	**FUNCTION**	**DATA**
Byte 6	Byte 7	Byte 8

As we can see from table 3, Modbus TCP/IP doesn't use error checking line in command sequence. Query-response corruption is detected using TCP/IP check mechanisms [13]. Modbus TCP/IP is utilizing Internet procotols, so device itentifiers and all needed parameters come from the server information. Let's go through the meaning of each line of command sequence.

Bytes 0-1: Transaction identifier

Recopied by the mainframe computer (DHCP server) from the received request.

Bytes 2-3: Protocol identifier

Bytes 4-5: Length

First byte is upper digit (max 256), second byte is lower digit which contains the number of following bytes

Byte 6: Modbus slave unit identifier (IP address)

Byte 7: Modbus function code

Byte 8: Implicates that the following byte are the data field

Modbus TCP/IP in data logger

Modbus TCP/IP was used for transfering data from Thermo Fisher analyzers to mainframe computer. Analyzer data is packed into 16-bit holding registers. The concentration data is 32 bits, so the data is split in two holding register addresses. Holding register data is in most significant bit (MSB) order, that is big-endian. This means that the highest value bits are presented in the first register holding the data and lowest value bits are in the second holding register.[3] This means that the register data is in unsigned integer, and it needs to be converted into 32-bit floating point number in the mainframe computer.

4 Discussion and Future Scheme

4.1 Automatic data logger

Through all the important tasks that has been done for the automation of atmospheric simulation chamber, there is still a lot to do. The automatic data logger is mostly done, but there is still some optimization that can be done for the data logging program.

As discussing of the data logger, all the Python functions that makes the data logger automatically log the data in database and text files, are in the same Python script. This could be optimized by saving the needed functions for a separate file and importing it the main script. This action helps the further work and might even prevent some bugs in the program. Naturally, there is always room for improvements but the program has been tested to run for a week and hasn't shown particular weaknesses.

4.2 Automatic process control

As discussed in section 1, automatic process control relies on the sensor data and mass flow controlling. There are some methods available for regulating pressure and RH based on sensor data. The intention is to use proportional integral derivative (PID) controllers to gain steady conditions inside the chamber. PID controller can be defined with formula in

$$
\begin{aligned}
PID &= K_P \cdot e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{de(t)}{dt} \\
e(t) &= v_s - v_t,
\end{aligned}
\tag{4.1}
$$

where K_P is scaling value for proportional term, K_I is scaling value for integral term, K_D is scaling value for derivative term, v_s is setpoint value and v_t is current value. By tuning the constant values K_P, K_I and K_D it is possible to stabilize the system into setpoint value. It can be done manually or using e.g. Ziegler–Nichols method. When the chamber conditions are stabilized, studies done inside it are more accurate and relevant for atmospheric science.

Other than stabilizing the conditions inside the chamber, there are few other improvements that can be made. Stabilizing the chamber conditions involves automatically controlling ozone generator. It finally gives the ideal conditions for studying atmospheric reactions of sample materials. Lastly, UV lights should be controlled remotely, as mentioned in section 1.

References

[1] Thermo Fisher Scientific. Model 42i-TL User Manual. https://documents.thermofisher.com/TFS-Assets/LSG/manuals/EPM-manual-Model%2042itl.pdf, 2007. Last visited 7.3.2026.

[2] Thermo Fisher Scientific. Model 43i-TLE User Manual. https://documents.thermofisher.com/TFS-Assets/LSG/manuals/EPM-manual-Model%2043i-tle.pdf, 2007. Last visited 7.3.2026.

[3] Thermo Fisher Scientific. Model 49i User Manual. https://documents.thermofisher.com/TFS-Assets/EPM/manuals/EPM-manual-Model%2049i.pdf, 2007. Last visited 7.3.2026.

[4] Advantech. ADAM-4510I, ADAM-4520I, ADAM-4117 Datasheet. https://advdownload.advantech.com/productfile/PIS/ADAM-4117/file/ADAM-4117_4510I_4520I_DS(011122)20220329095946.pdf. Last visited 7.3.2026.

[5] Advantech. ADAM-4520/S, ADAM-4520, ADAM-4521 Datasheet. https://advdownload.advantech.com/productfile/PIS/ADAM-4520/file/ADAM-4510_S_4520_4521_DS20210326145943.pdf. Last visited 7.3.2026.

[6] Vaisala Oyj. Product documentation portal — docs.vaisala.com. https://docs.vaisala.com/r/M211060EN-N/en-US. Last visited 7.3.2026.

[7] M. Cotton, L. Eggert, J. Touch, M. Westerlund, and S. Cheshire. Internet Assigned Numbers Authority (IANA) Procedures for the Management of the Service Name and Transport Protocol Port Number Registry. 2011. Last visited 7.3.2026.

[8] Maker.io. Understanding the RS-232 standard. https://www.digikey.fi/en/maker/blogs/2023/understanding-the-rs-232-standard, 2023. Last visited 7.3.2026.

[9] Asynchronous Serial Communication - an overview | ScienceDirect Topics — sciencedirect.com. https://www.sciencedirect.com/topics/computer-science/asynchronous-serial-communication#:~:text=Asynchronous%20serial%20communication%20is%20a,signal%20between%20sender%20and%20receiver. Last visited 7.3.2026.

[10] Maker.io. What is RS-485? https://www.digikey.fi/fi/maker/tutorials/2023/what-is-rs-485, 2023. Last visited 7.3.2026.

[11] Brainboxes. ASCII Protocol and Commands - Brainboxes — brainboxes.com. https://www.brainboxes.com/faq/ascii-protocol-and-commands. Last visited 7.3.2026.

[12] Alicat Scientific. Serial Communication Tutorial. https://www.alicat.com/support/serial-communication-tutorial/. Last visited 7.3.2026.

[13] MODICON Inc. Modbus protocol reference guide. https://www.mgco.jp/mssenglish/PDF/EM/service/emmodbus.pdf, 1996. Last visited 7.3.2026.

[14] Advantech. ADAM-4100 series User Manual. https://advdownload.advantech.com/productfile/Downloadfile1/1-33T3L5U/ADAM-4100_User_Manual_Ed.7-FINAL.pdf. Last visited 7.3.2026.